

Chapter 7

Chapter

- 1 Project mgmt concepts
- 2 Project planning, control and process model
- 3 Quality mgmt and assurance
- 4 Software metrics
- 5 Risk mgmt
- 6 Software process improvement

Software Configuration Management

Table of Contents

7.1 **Software Configuration Management (SCM)**

- What is it?
- What are the SCM activities?

7.2 **Version** Management

7.3 **System** Building

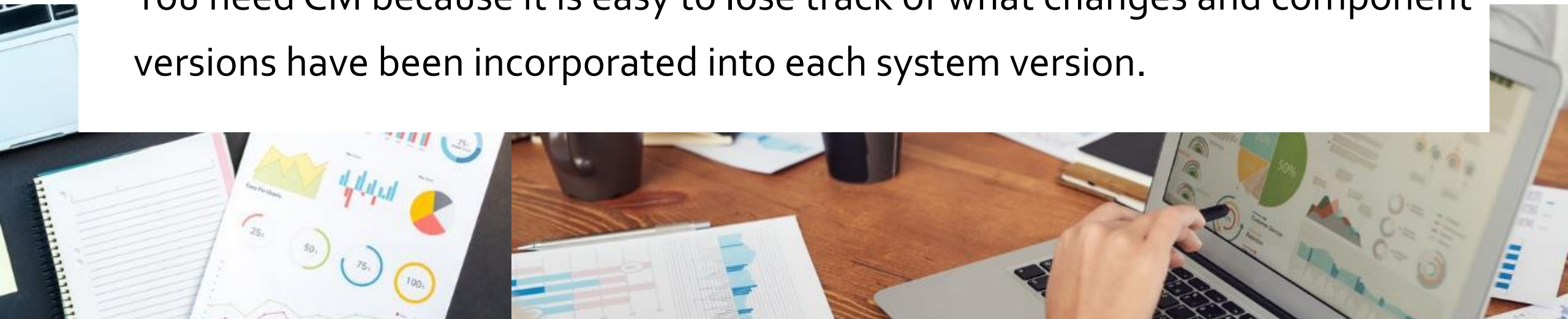
7.4 **Release** Management

7.5 **Change** Management



7.1 What Software Configuration Management (SCM) is all about?

- **Software systems will change during development and use.** SCM or CM is about managing changes to software systems by establishing policies, processes and tools.
- **SCM/CM is to control changes** so that everything will work smoothly and you can always go back to a particular version if needed.
- You need CM because it is easy to lose track of what changes and component versions have been incorporated into each system version.



7.1 Configuration Management - Activities



Version Management

Keeping track of the multiple versions of system components and ensuring that changes made to components by different developers do not interfere with each other.



System Building

The process of assembling program components, data and libraries, then compiling these to create an executable system.



Release Management

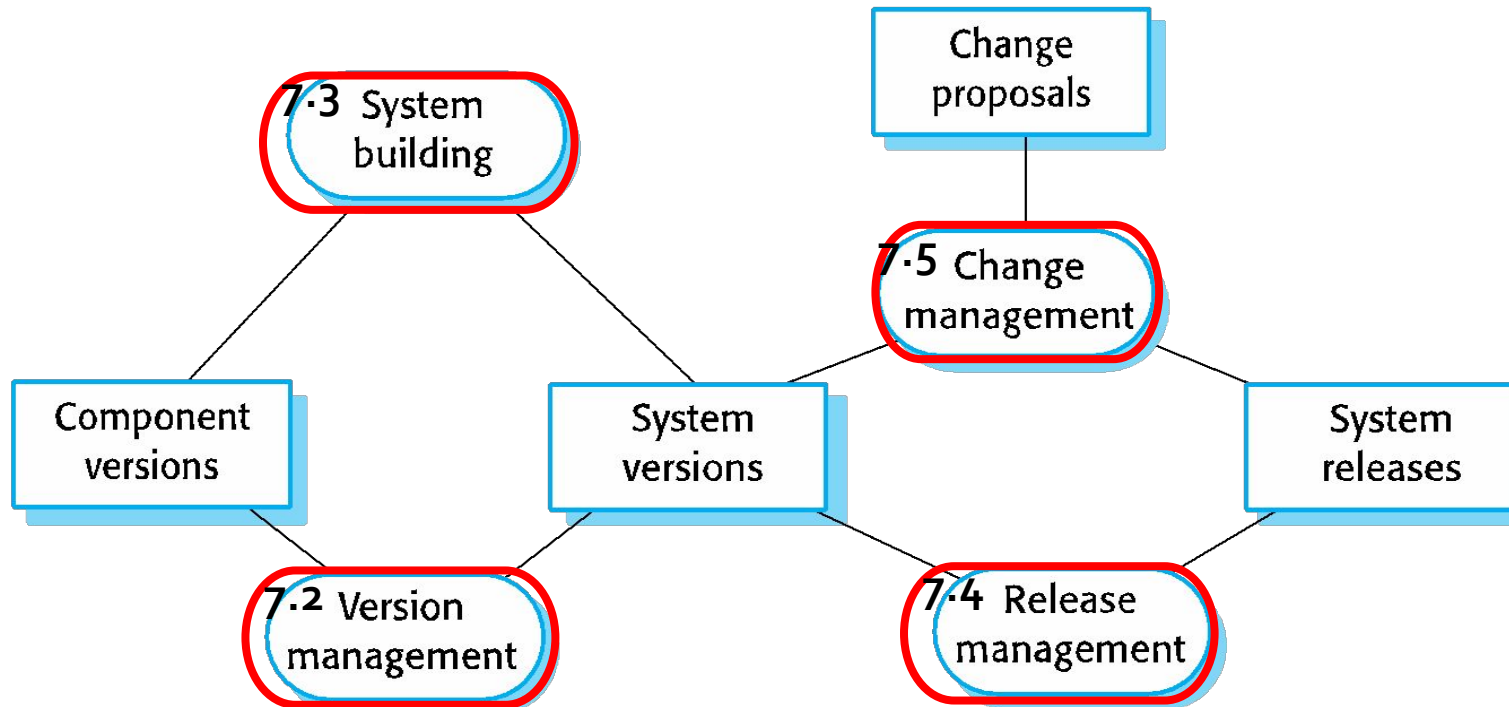
Preparing software for external release and keeping track of the system versions that have been released for customer use.



Change Management

Keeping track of requests for changes to the software from customers and developers, working out the costs and impact of changes, and deciding the changes should be implemented.

7.1 Configuration Management - Activities



Software systems will change during development and use.
The above activities are needed to ...

Configuration Management at Different Phases

7.1 Configuration Management

Possible changes to software during development:

Adding a **new** functionality/feature. *(e.g.1 adding "finger print" feature to log-in
(e.g.2 adding payment using e-wallet
(e.g.3 _____*

In the testing phase, *changes to software could be due to:*

- Bugs *fixes*
- Performance enhancement
- Security repairs, etc

Changes (if any) during DEVELOPMENT and USE must be properly **track, control and documented.** _____



Configuration Management Terminology

7.1 Configuration Management

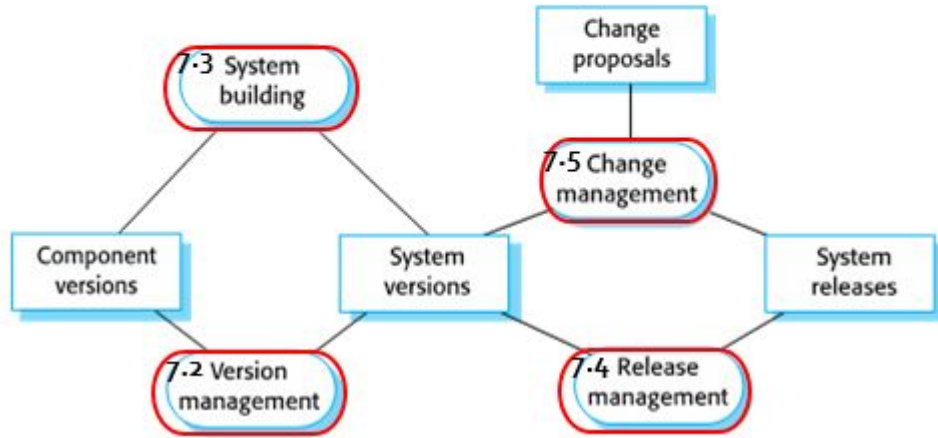
Term	Explanation
Software Configuration Item (SCI) or C.I.	Anything associated with a software proj (e.g. design spec, source code, test data, user manual, etc.) that has been placed under configuration control. There are often different versions of a configuration item. Configuration items have a unique name.
Version	An instance of a SCI or C.I. that differs, in some way, from other instances of that item. Versions always have a unique identifier. (e.g. Python Login Source Code user_authentication_v1.0.py (login with userID & pw) What is the difference between user_authentication_v1.1.py (added login using gmail) v1.0 and v1.1? _____
Codeline	A sequence of versions of source code with later versions in the sequence derived from earlier versions.
Baseline	A collection of <u>component versions</u> that make up a system. Baselines are controlled, which means that the versions of the components making up the system cannot be changed. This means that it is always possible to recreate a baseline from its constituent components.
Mainline	A sequence of <u>baselines</u> representing different versions of a system.

Configuration Management Terminology (con't)

7.1 Configuration Management

Term	Explanation
Branching	The creation of a new codeline from a version in an existing codeline. The new codeline and the existing codeline may then develop independently.
Merging	The creation of a new version of a software component by merging separate versions in different codelines. These codelines may have been created by a previous branch of one of the codelines involved.
Configuration (version) control	The process of ensuring that versions of systems and components are recorded and maintained so that changes are managed and all versions of components are identified and stored for the lifetime of the system.
Release	A version of a system that has been released to customers (or other users in an organization) for use.
Repository	A shared database of versions of software components and meta-information about changes to these components.
System building	The creation of an executable system version by compiling and linking the appropriate versions of the components and libraries making up the system.
Workspace	A private work area where software can be modified without affecting other developers who may be using or modifying that software.

7.1 Configuration Management - Activities



7.2 Version Management

- Keep track of different versions of software components or configuration items and the systems that uses these components.
- It also involves ensuring that changes made by different developers to these versions do not interfere with each other.
- **Version management** can be thought of as the process of managing codelines and baselines.

is a sequence of versions of source code

is a definition of a specific system.
(i.e. what components are included in a version of a particular system)

Codelines and Baselines

7.2 Version Management

Codeline

- A codeline is a sequence of versions of source code with later versions in the sequence derived from earlier versions.
- Codelines normally apply to components of systems so that there are different versions of each component.

Baseline

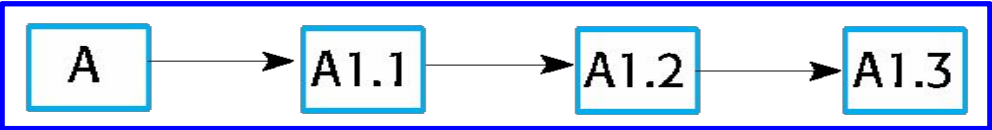
- A baseline is a definition of a specific system.
- The baseline therefore specifies the component versions that are included in the system plus a specification of the libraries used, configuration files, etc.
- Baselines may be specified using a configuration language, which allows you to specify what components are included in a version of a particular system.
- Baselines are important because you often have to recreate a specific version of a complete system.
- For example, a product line may be instantiated so that there are individual system versions for different customers. You may have to recreate the version delivered to a specific customer if, for example, that customer reports bugs in their system that have to be repaired.

Codelines and Baselines

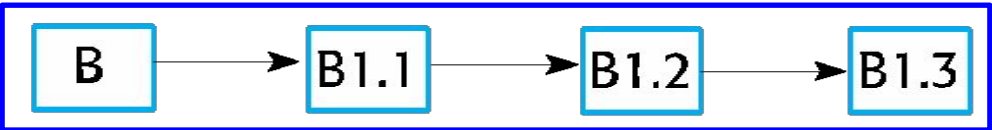
7.2 Version Management

Codeline = Sequence of versions of source code

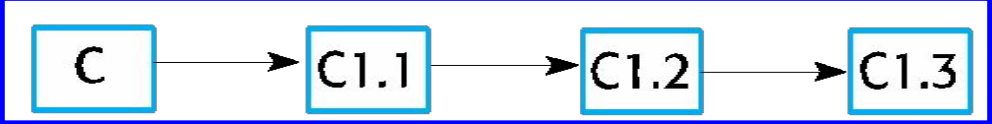
Codeline (A)



Codeline (B)



Codeline (C)



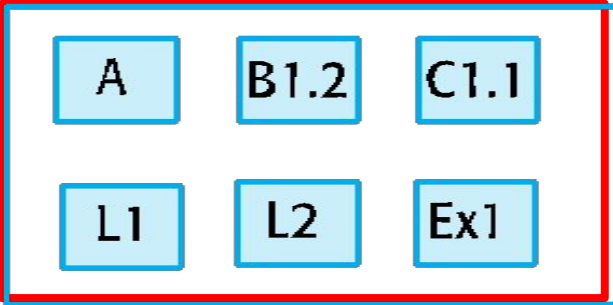
Libraries and external components



Baseline = Collection of components of a system.

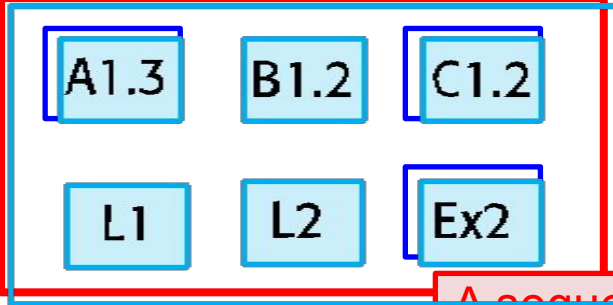
e.g. V1 of **system XYZ** (consists of ...)

Baseline - V1



V2 of **system XYZ** (consists of ...)

Baseline - V2



Mainline

A sequence of baselines e.g. XYZ.V1, XYZ.V2 representing different versions of a system

Configuration Management Terminology

7.1 Configuration Management

Quick Review...

Term	Explanation
Software configuration item (SCI)	Anything associated with a software proj (<u>e.g.</u> design spec, source code, test data, user manual, etc.) that has been placed under configuration control. There are often different versions of a configuration item. Configuration items have a unique name.
Version	An instance of a software configuration item (or configuration item) that differs, in some way, from other instances of that item. Versions always have a unique identifier.
Codeline	A sequence of versions of source code with later versions in the sequence derived from earlier versions.
Baseline	A collection of component versions that make up a system. Baselines are controlled, which means that the versions of the components making up the system cannot be changed. This means that it is always possible to recreate a baseline from its constituent components.
Mainline	A sequence of baselines representing different versions of a system.



Version Control Systems

7.2 Version Management

- Is a system that **identify, store and control** access to the different versions of components.
- **2 types of modern version control system:**

Centralized version control systems

There is a single master repository that maintains all versions of the software components that are being developed. **E.g: Subversion**

Distributed version control systems

Multiple versions of the component repository exist at the same time. **E.g: Git**



Features of VC Systems

7.2 Version Management

- Version and release identification
- Change history recording
- Support for independent development
- Project support
- Storage management



Public Repository & Private Workspaces

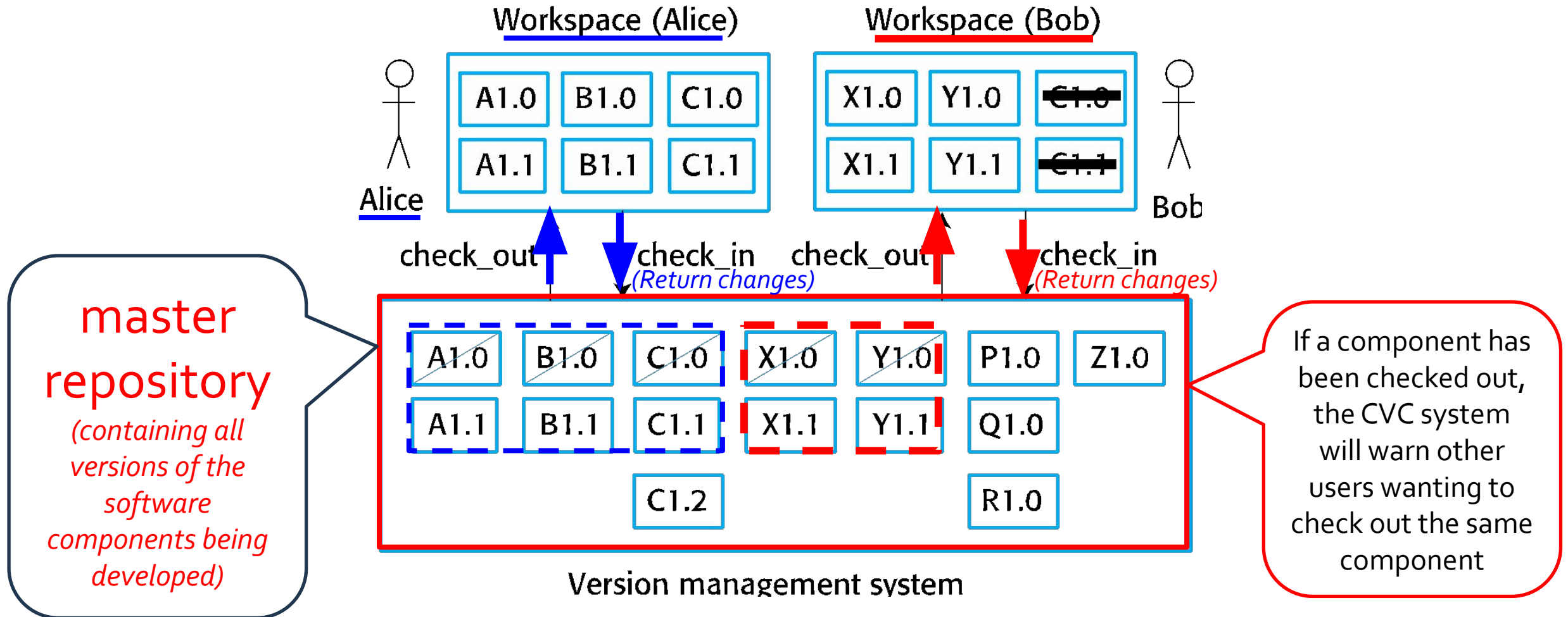
7.2 Version Management

- To support independent development without interference, version control systems use the concept of a project repository and a private workspace.
- The project repository maintains the 'master' version of all components. It is used to create baselines for system building.
- When modifying components, developers copy (check-out) these from the public repository into their private workspace and work on these copies.
- When they have finished their changes, the changed components are returned (checked-in) to the public repository.

Centralized Version Control

7.2 Version Management

(An example of a widely used CVC is **Subversion**)





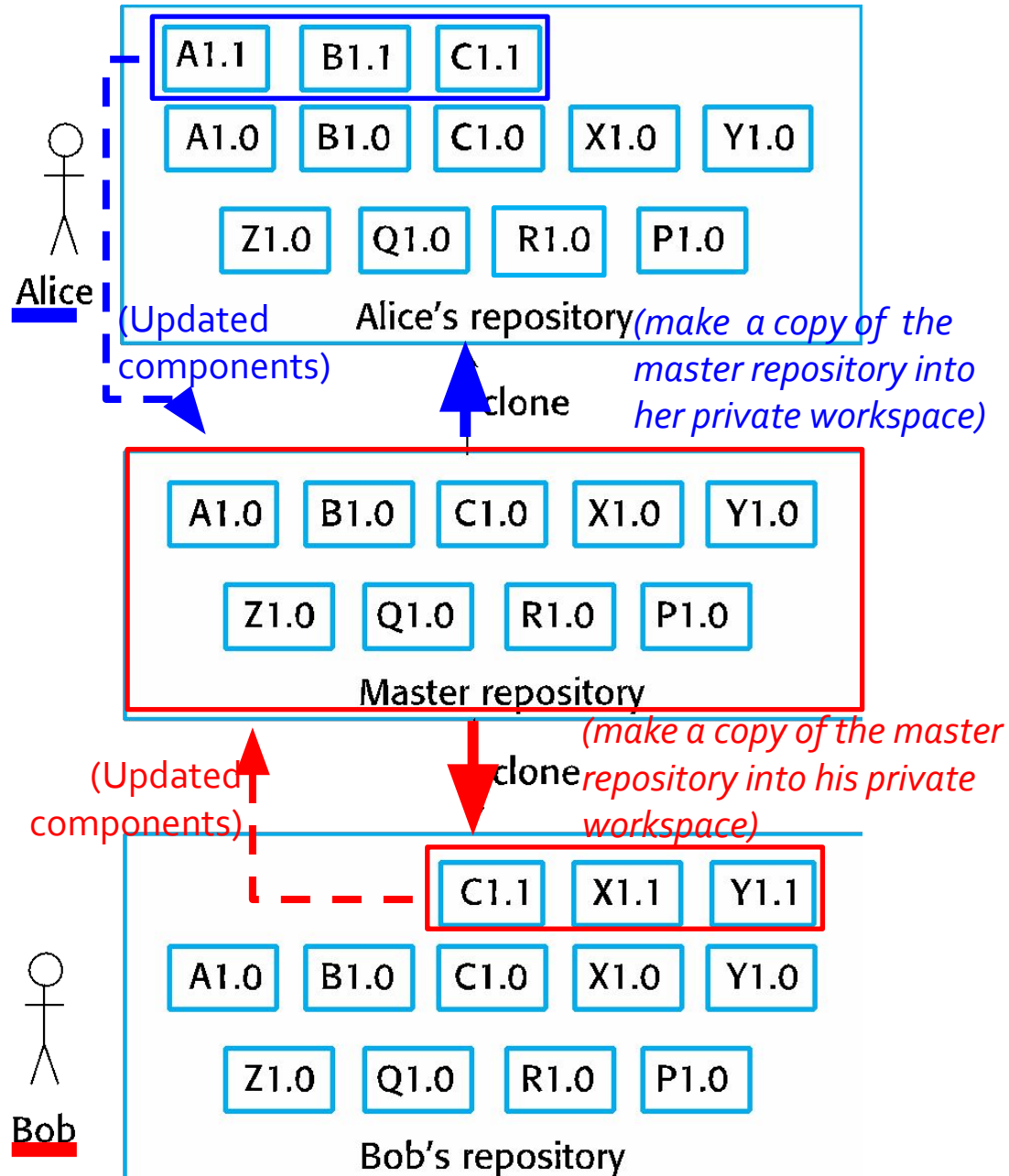
Distributed Version Control

7.2 Version Management

- A 'master' repository is created on a server that maintains the code produced by the development team.
- Instead of checking out the files that they need, a developer creates a clone of the project repository that is downloaded and installed on their computer.
- Developers work on the files required and maintain the new versions on their private repository on their own computer.
- When changes are done, they 'commit' these changes and update their private server repository. They may then 'push' these changes to the project repository.

Distributed Version Control

(An example of a widely used DVC is **Git**)



Benefits of Distributed Version Control

- **It provides a backup mechanism for the repository.**
 - If the repository is corrupted, work can continue and the project repository can be restored from local copies.
- **It allows for off-line working.**
- **Project support is the default way of working.**
 - Developers can compile and test the entire system on their local machines and test the changes that they have made.

Branching and Merging

7.2 Version Management

- Rather than a linear sequence of versions that reflect changes to the component over time, there may be several independent sequences.
 - This is normal in system development, where different developers work independently on different versions of the source code and so change it in different ways.
- At some stage, it may be necessary to merge codeline branches to create a new version of a component that includes all changes that have been made.
 - If the changes made involve different parts of the code, the component versions may be merged automatically by combining the deltas that apply to the code.

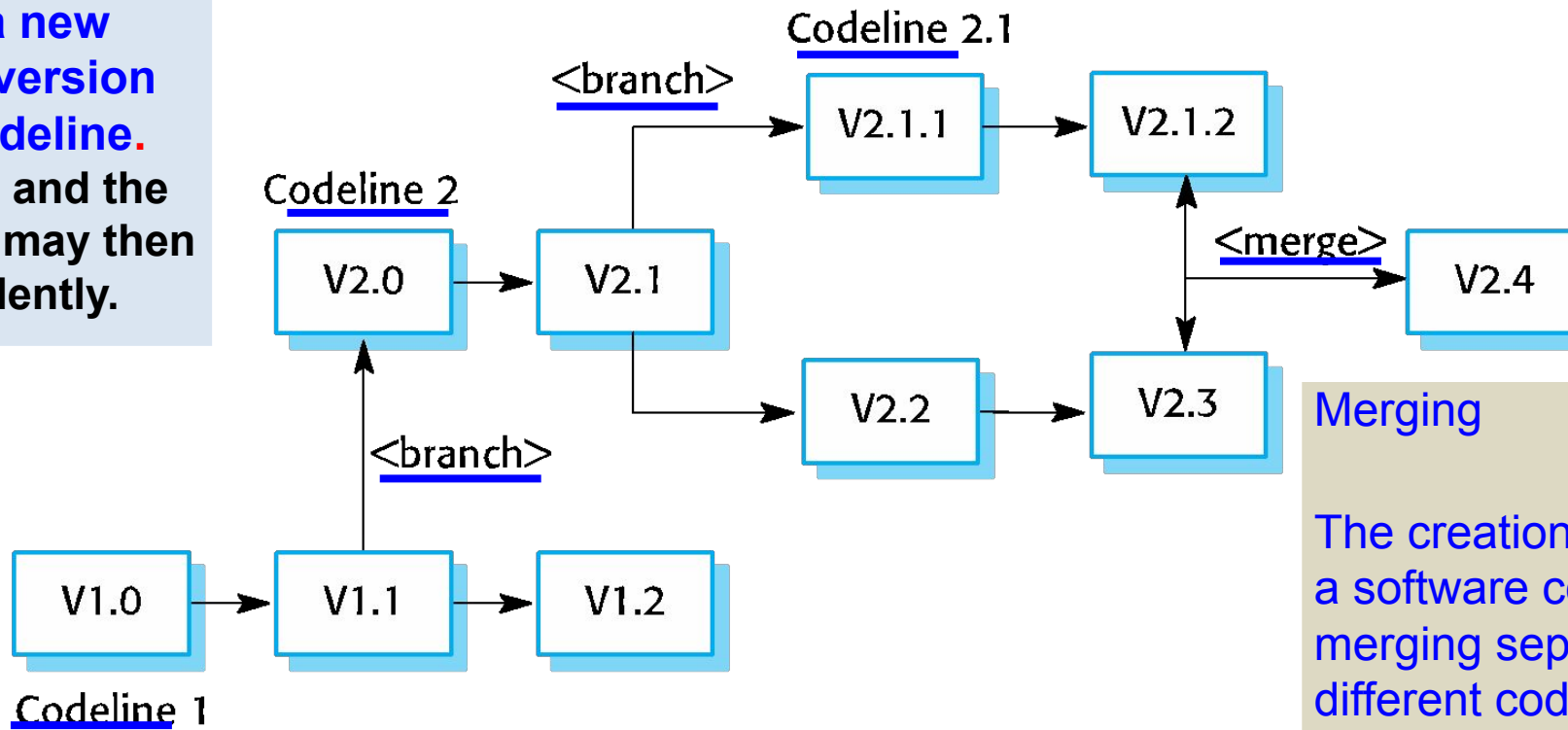
Branching and Merging

7.2 Version Management

Branching

The creation of a new codeline from a version in an existing codeline.

The new codeline and the existing codeline may then develop independently.



Sequence of versions of source code

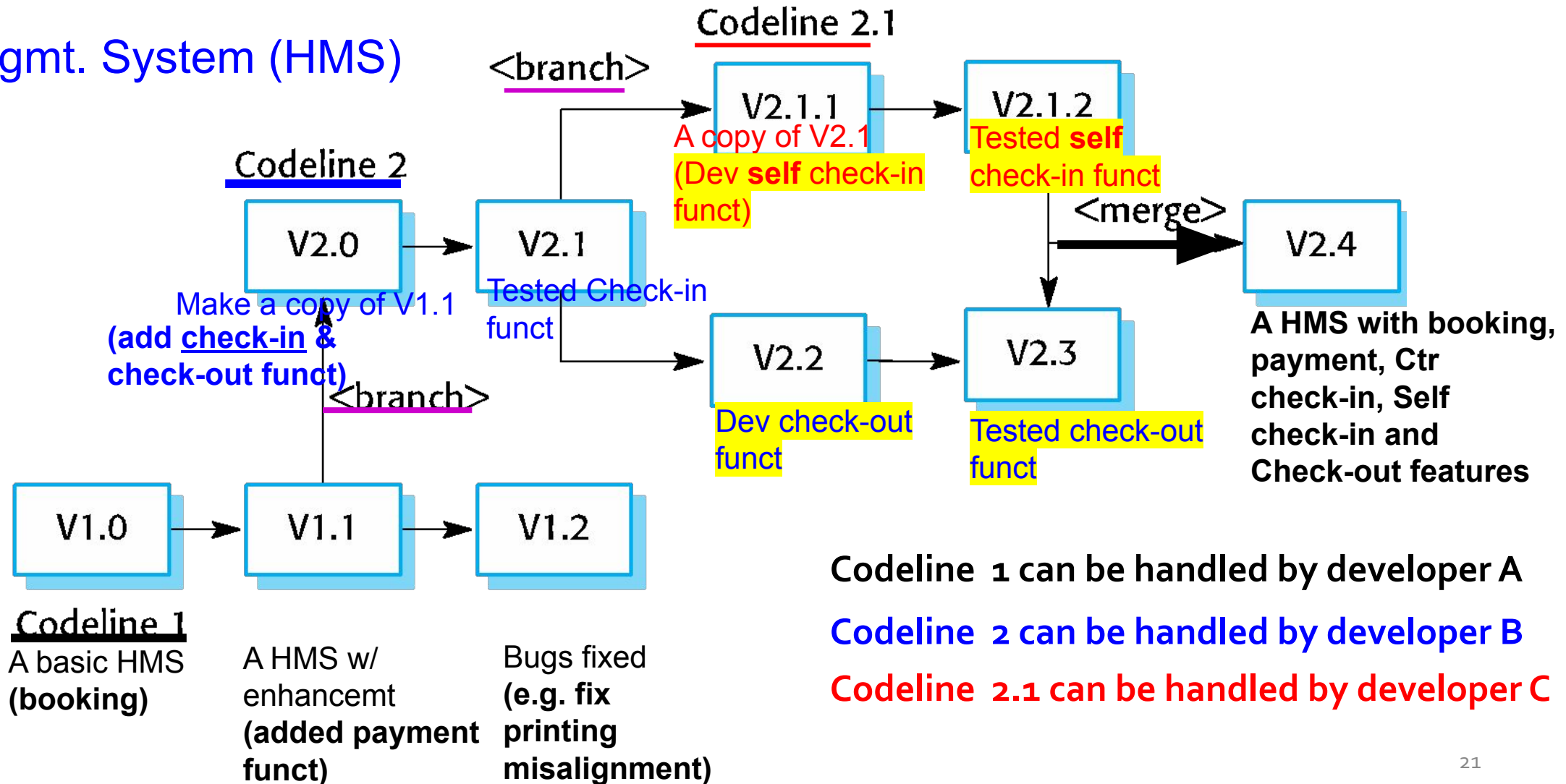
Merging

The creation of a new version of a software component by merging separate versions in different codelines. These codelines may have been created by a previous branch of one of the codelines involved.

Branching and Merging

7.2 Version Management

Hotel Mgmt. System (HMS)





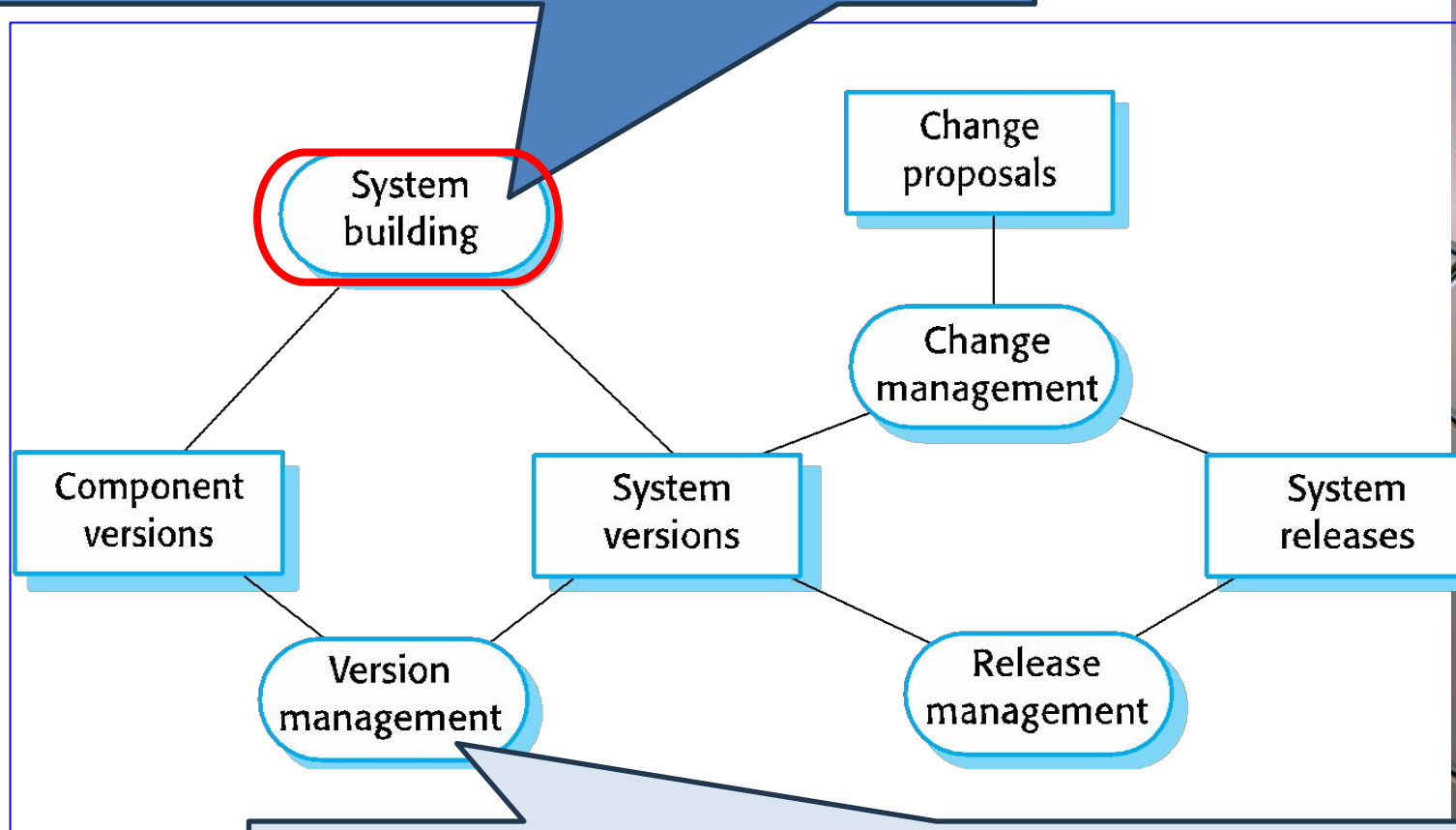
Storage Management

7.2 Version Management

- When version control systems were first developed, storage management was one of their most important functions.
- Disk space was expensive and it was important to minimize the disk space used by the different copies of components.
- Instead of keeping a complete copy of each version, the system stores a list of differences (deltas) between one version and another.
- By applying these to a master version (usually the most recent version), a target version can be recreated.

7.3 System Building

7.3 System Building is the process of **assembling program components, data and libraries**, then compiling these to create an executable system.



- System building tools and version management tools must communicate as the build process involves checking out component versions from the repository managed by the version management system.
- The configuration description used to identify a baseline is also used by the system building tool.

(7.2) VM is the process of keeping track of different versions of **configuration items** and the systems that uses these components.

Build System Platforms

7.3 System Building

Development System

Includes development tools such as compilers, source code editors, etc.

Developers check out code from the version management system into a private workspace before making changes to the system.

Build Server

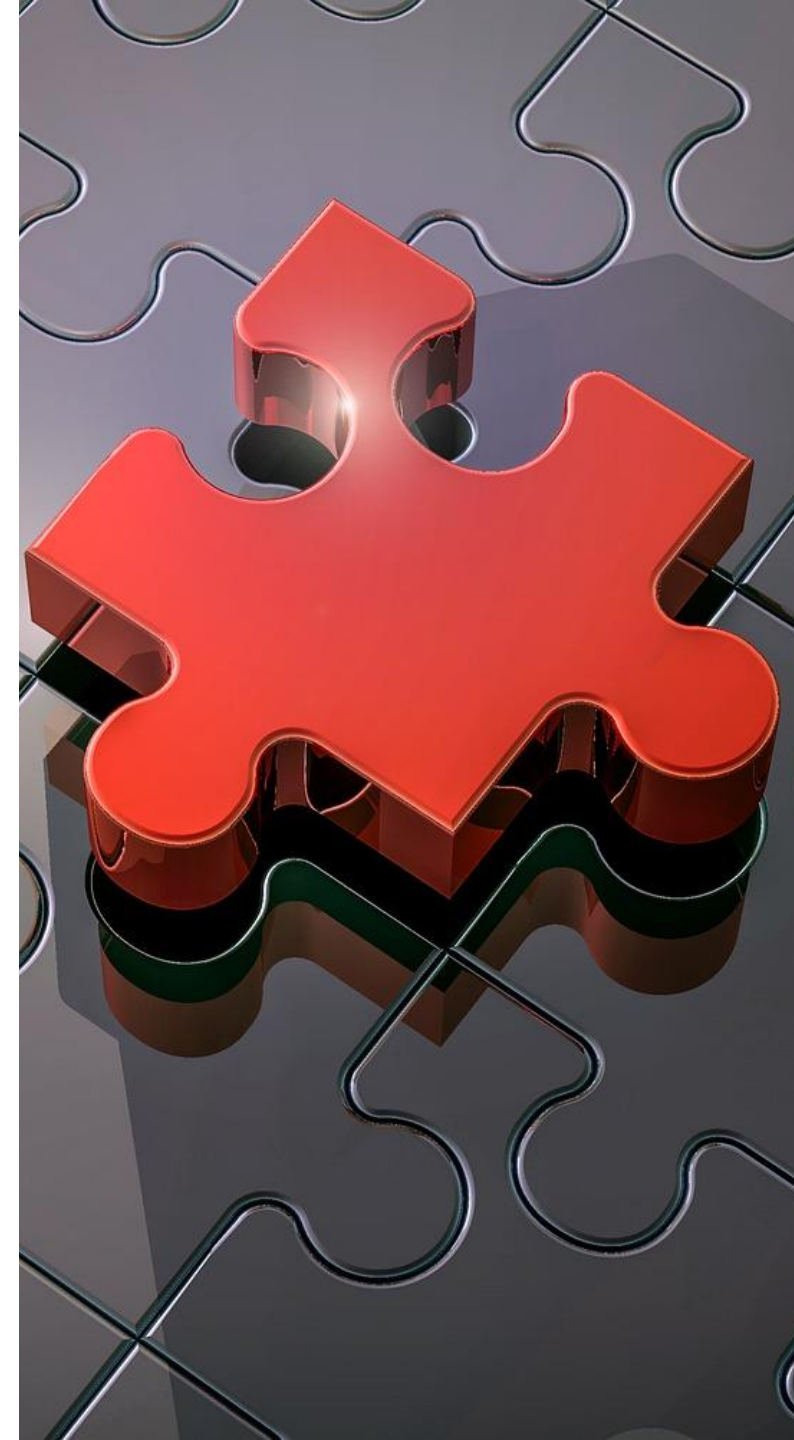
Used to build definitive, executable versions of the system.

Developers check-in code to the version management system before it is built. The system build may rely on external libraries that are not included in the version management system.

Target Environment

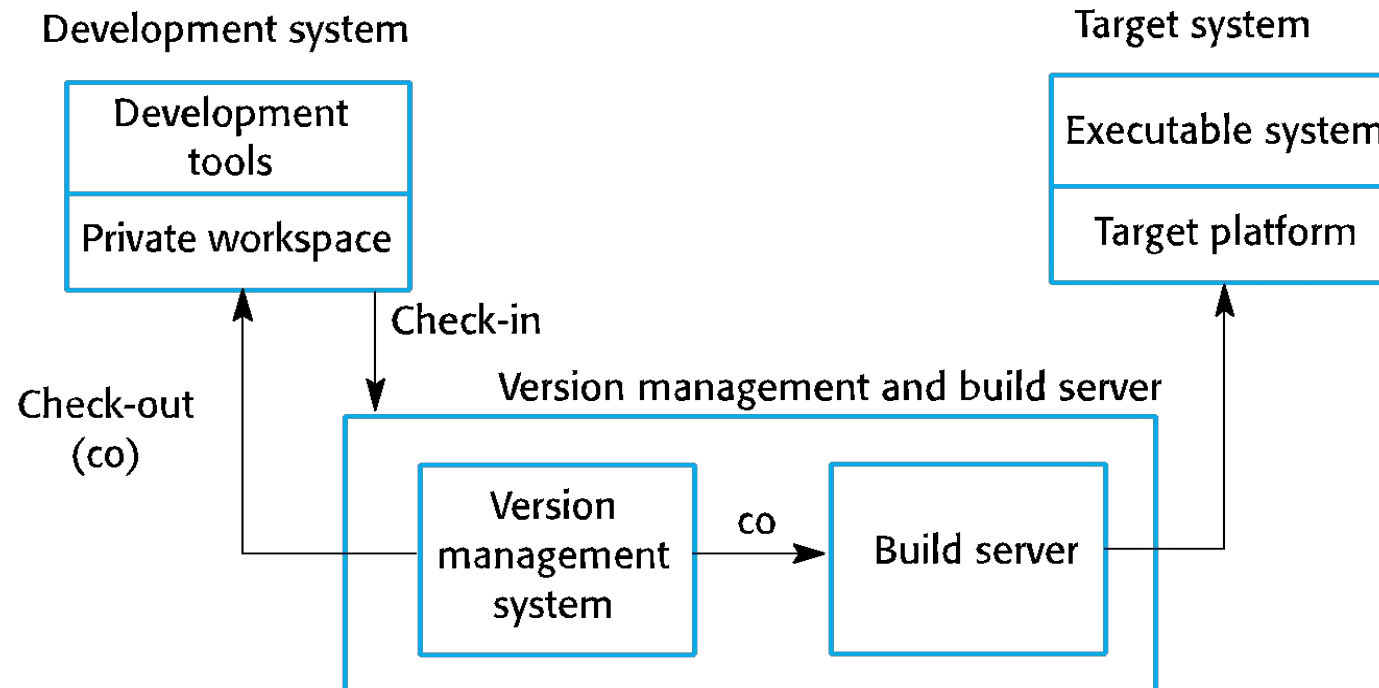
The platform on which the system executes.

For real-time and embedded systems, the target environment is often smaller and simpler than the development environment (e.g. a cell phone)



Build System Platforms

7.3 System Building

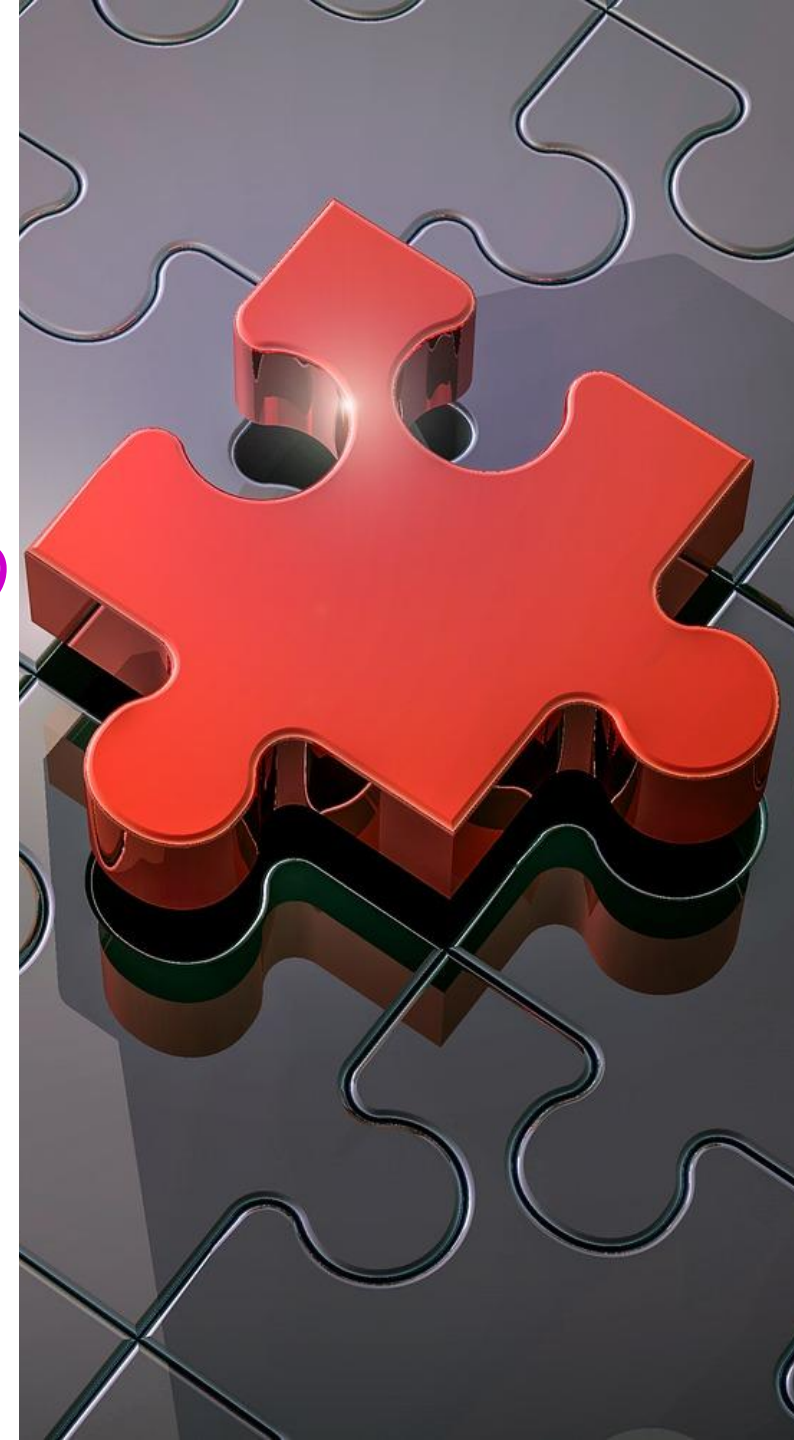
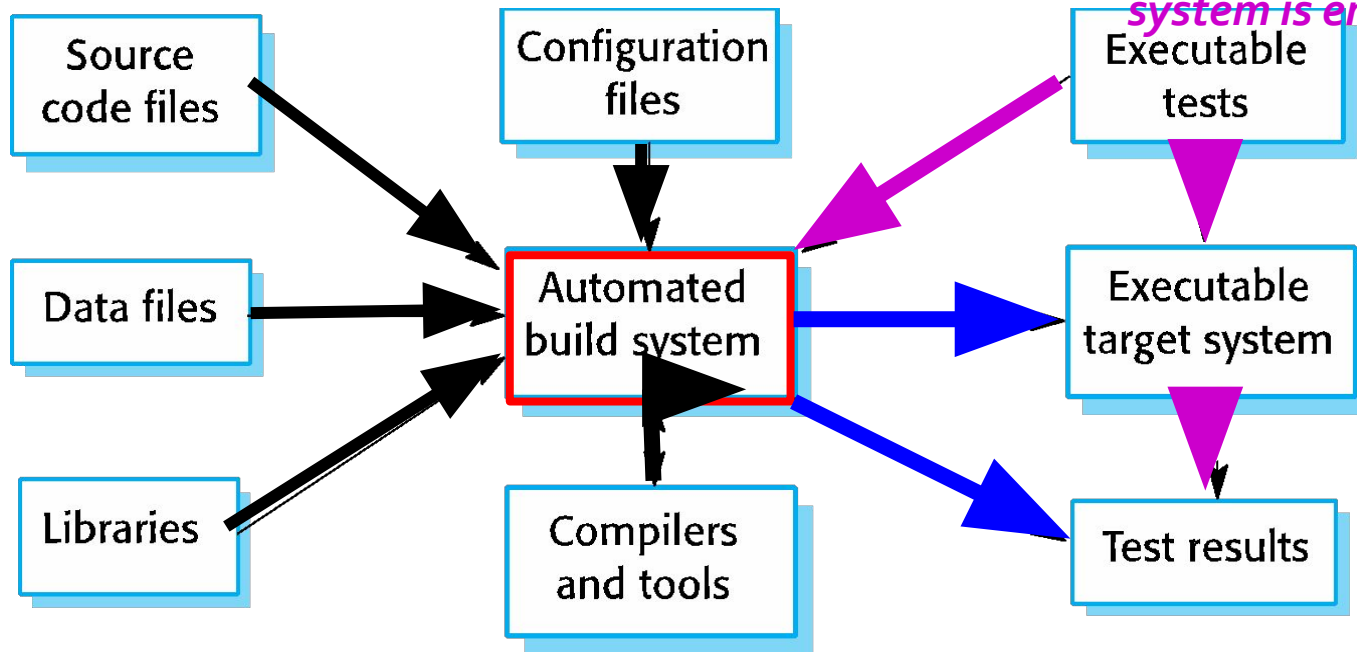


7.3 System Building

1. Gather
Input

2. Produce
Compilation report
& Target System

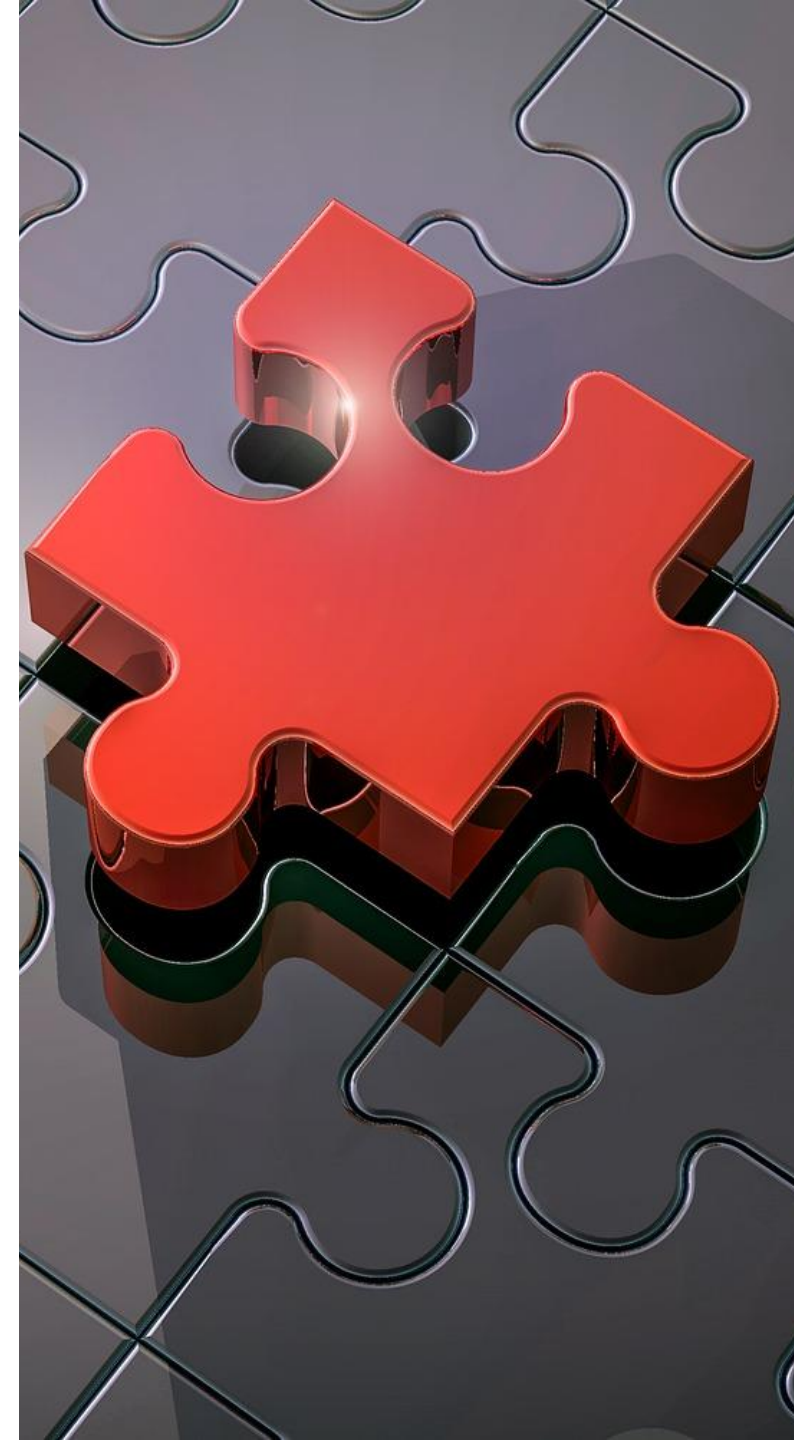
3. Run
automated
Test (to confirm
system is error free)



Build System Functionality

7.3 System Building

- Build script generation
- Version management system integration
- Minimal re-compilation
- Executable system creation
- Test automation
- Reporting
- Documentation generation



Daily Building Example

7.3 System Building

- The development organization sets a delivery time (say 2 p.m.) for system components.
 - If developers have new versions of the components that they are writing, they must deliver them by that time.
 - A new version of the system is built from these components by compiling and linking them to form a complete system.
 - This system is then delivered to the testing team, which carries out a set of predefined system tests
 - Faults that are discovered during system testing are documented and returned to the system developers. They repair these faults in a subsequent version of the component.



7.4 Release Management

- Process of preparing software for distribution to the customers and keeping track of the versions that have been released.
- **For the mass market**, it is usually possible to identify **2 types of release**:
 - **minor release**: delivery of software after fixing bugs and resolving customer problems/complaints *E.g Win8 □ Win8.1*
 - **major release**: To deliver significantly new functionality
E.g Win7 □ Win8 □ Win10 □ Win11
- **For custom software**, releases of the system may have to be produced for each customer and individual customers may be running different releases of the system at the same time.

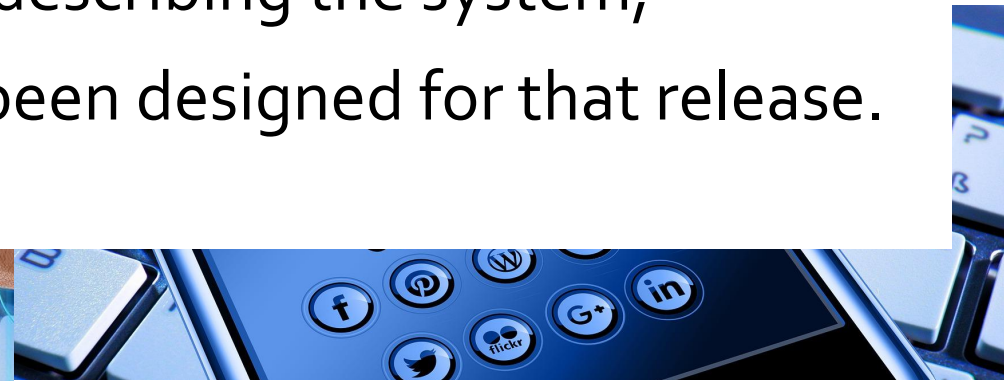
	V1.0	V1.1
KK City Hotel	Room browsing, chk-in & chk-out, pymt	Room browsing, chk-in & chk-out, pymt and Room service
SP Airport Hotel	Room browsing, chk-in & chk-out, pymt	

Release Components

7.5 Release Management

Components in a release:

- **Executable code** of the system,
- **Configuration files** defining how the release should be configured for particular installations;
- **Data files**, such as files of error messages, that are needed for successful system operation;
- **Installer** - program that is used to help install the system on target hardware;
- **Documentation (softcopy and/or hardcopy)** describing the system;
- packaging and associated publicity that have been designed for that release.



Factors Influencing System Release

7.5 Release Management

Factor	Description
Competition	For mass-market software, when a competing product has introduced <u>new features</u> market share may be lost if a new system release is not provided to your existing customers.
Platform changes	When a new version of the OS platform is released, you may have to create a new release of the software application for the updated OS (E.g: Microsoft upgraded Windows 10 to Windows 11. Your game software also must be updated to make it compatible with Windows 11. Else old game software will not function smoothly)
Technical quality of the system	When there are serious system faults which affect many customers, it may be necessary to issue a fault repair release. Minor system faults may be repaired by issuing patches (usually distributed over the Internet) that can be applied to the current release of the system.
Marketing requirements	The marketing department of an organization have made a commitment for releases to be available at a particular date.

7.5 Change Management

- Organizational needs and requirements change during the lifetime of a system, bugs have to be repaired and systems have to adapt to changes in their environment.
- **Software Configuration Mgmt (SCM) is aka *Change Management(CM)*.** Change management is to ensure that system evolution is **properly managed** and that priority is given to the most urgent and cost-effective changes.
- The change management process is concerned with analyzing the costs and benefits of proposed changes, **approving those changes that are worthwhile and tracking** which components in the system have been changed.

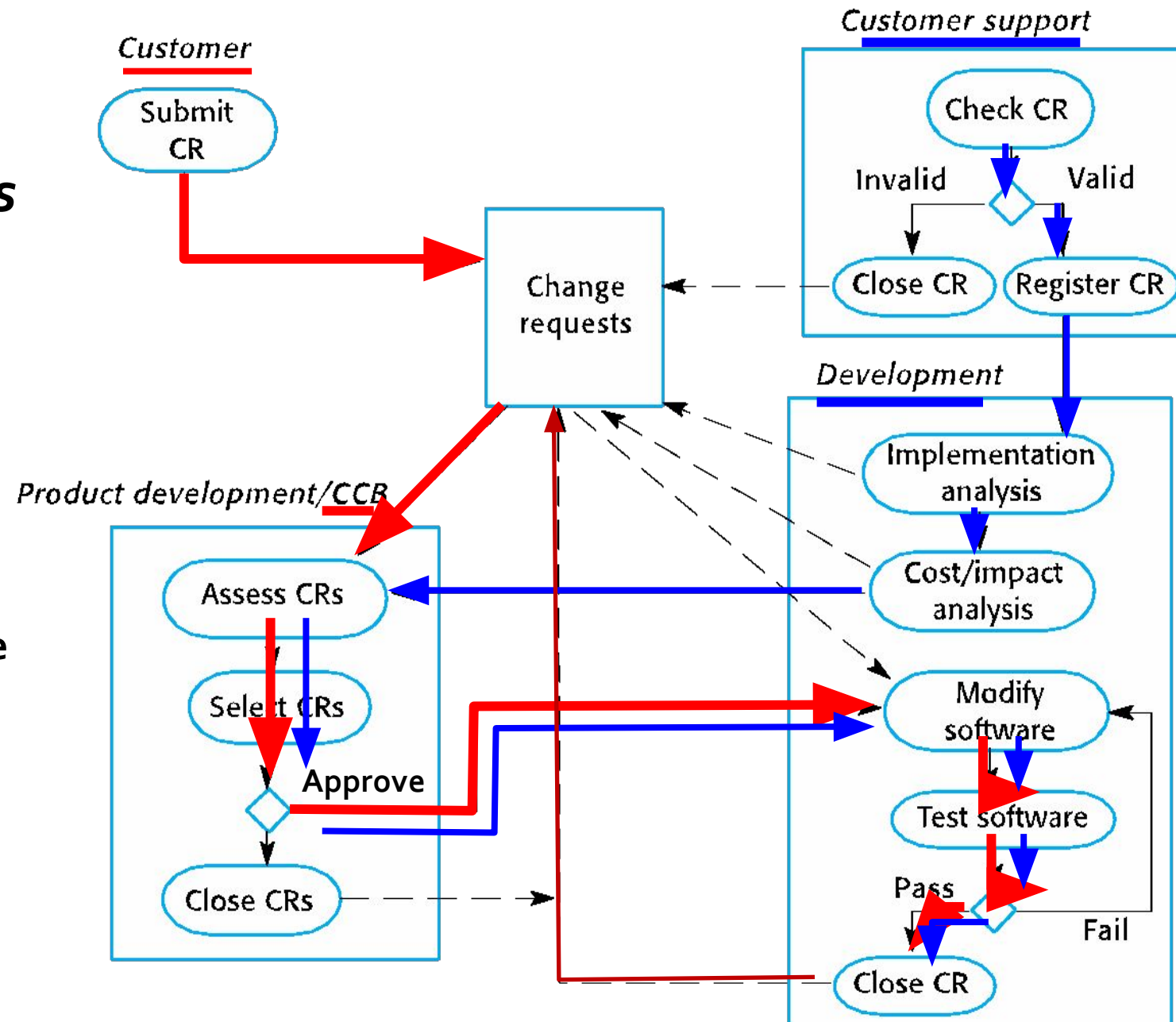


The *change management process*

(CR = Change Request)

Evaluating Change Request:

- The **benefits** of the change
- The **consequences** of not making the change
- The **costs** of making the change
- The **number of users** affected by the change
- The product release cycle



Sample Change Request form

Change Request Form

Project/System: SICSA/AppProcessing **Number:** 23/02

Change requester: I. Sommerville **Date:** 20/07/12

Requested change: ~~The status of applicants (rejected, accepted, etc.) should be shown visually in the displayed list of applicants. ...~~

Change analyzer: R. Looek **Analysis date:** 25/07/12

Components affected: ApplicantListDisplay, StatusUpdater

Associated components: StudentDatabase

Change assessment: Relatively simple to implement by changing the display color according to status. A table must be added to relate status to colors. No changes to associated components are required.

Change priority: Medium

Change implementation:

Estimated effort: 2 hours

Date to SGA app. team: 28/07/12 **CCB decision date:** 30/07/12

Decision: Accept change. Change to be implemented in Release 1.2

Change implementor: **Date of change:**

Date submitted to QA: **QA decision:**

Date submitted to CM:

Comments:

Summary

SCM/CM is about **controlling changes** during software development and while it is being used. All changes to **SCI** must be properly track, control and documented.

7.1 Software Configuration Management

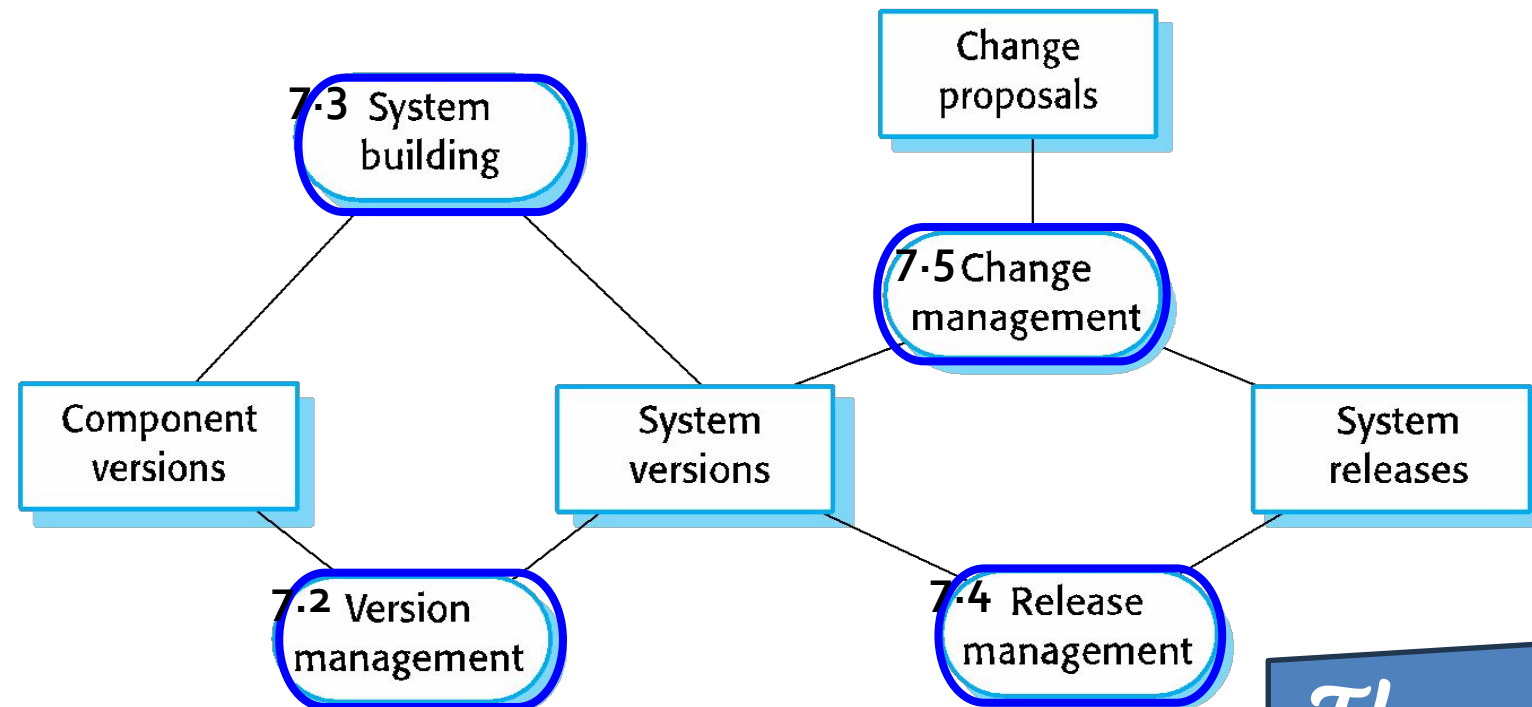
7.2 Version Management

7.3 System Building

7.4 Release Management

7.5 Change Management

Configuration Management Activities



The end ...